16.4-AI-ASS
2303A52249
A.VAMSHI KRISHNA

Task:01

Database Schema Design for an Online Course Platform
Prompt: You are designing the backend database for an online learning platform that manages instructors, courses, and enrolled students.

Task Description:

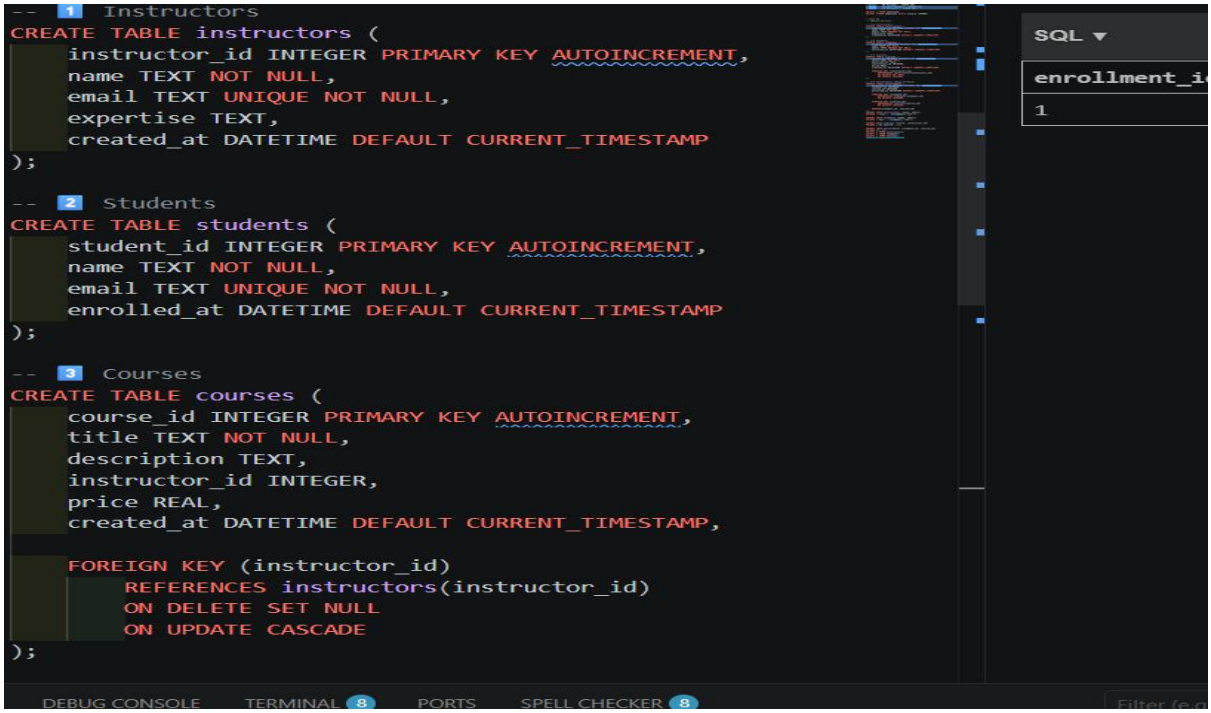
• Use AI tools to generate SQL CREATE TABLE statements for the following entities:

o Instructors, Courses, Students

• Include: Primary keys, Foreign key relationships, Appropriate data types

• Ensure the schema follows basic normalization principles.

Code:



```sql
-- 4 Enrollments (Many-to-Many)
CREATE TABLE enrollments (
    enrollment_id INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id INTEGER,
    course_id INTEGER,
    enrolled_on DATETIME DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (student_id)
        REFERENCES students(student_id)
        ON DELETE CASCADE,

    FOREIGN KEY (course_id)
        REFERENCES courses(course_id)
        ON DELETE CASCADE,

    UNIQUE(student_id, course_id)
);
```

Output:

student_id	name	email	enrolled_at
1	Sai	sai@gmail.com	2026-03-26 09:03:42

Observation:

The database schema is well-structured and normalized, with separate tables for instructors, courses, and students. Primary keys ensure unique records, while foreign keys maintain valid relationships between tables. Appropriate data types and constraints improve data integrity. Overall, the design is efficient, scalable, and follows basic normalization principles.

Task:02

Populate the Database with Sample Records

Prompt: To test the system, the platform needs initial sample data.

Task Description:

• Use AI assistance to generate INSERT statements that add:

o At least 3 instructors, o At least 4 courses, o At least 3 students

• Verify correctness using SELECT queries.

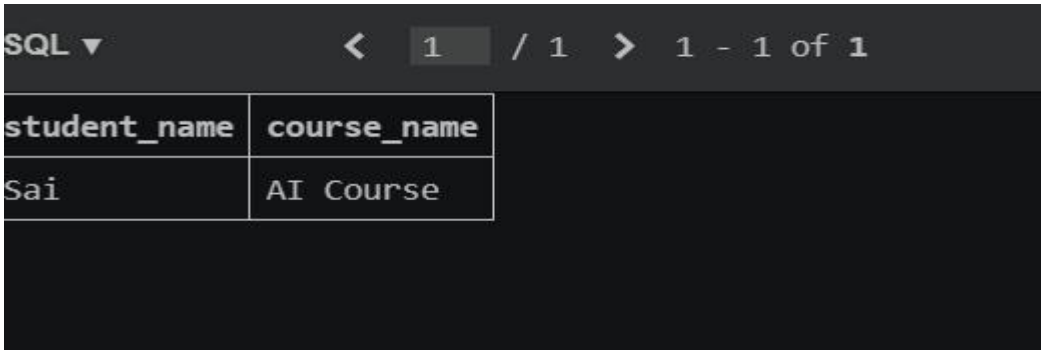
Code:

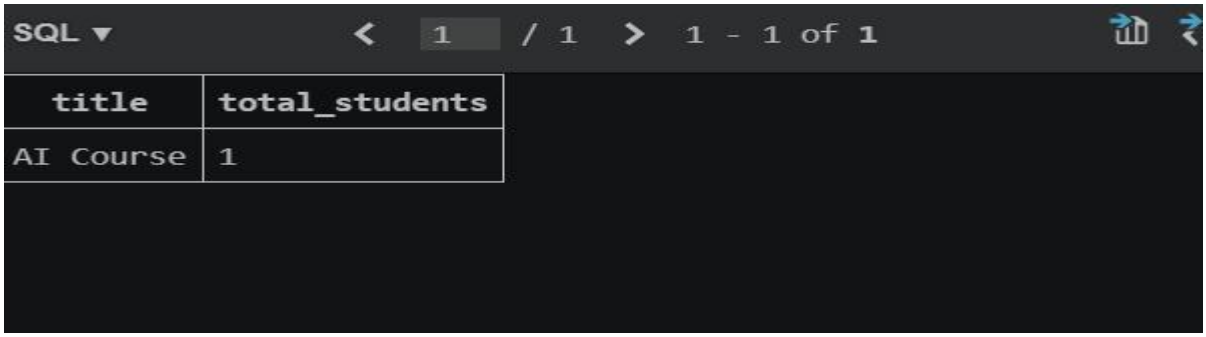
```sql
--task 02
SELECT
    c.course_id,
    c.title,
    i.name AS instructor_name
FROM courses c
JOIN instructors i
ON c.instructor_id = i.instructor_id;
SELECT
    s.name AS student_name,
    c.title AS course_name
FROM enrollments e
JOIN students s
ON e.student_id = s.student_id
JOIN courses c
ON e.course_id = c.course_id;
SELECT
    c.title,
    COUNT(e.student_id) AS total_students
FROM courses c
LEFT JOIN enrollments e
ON c.course_id = e.course_id
GROUP BY c.course_id;
SELECT *
FROM courses
WHERE price > 800;
SELECT *
FROM students
WHERE name = 'Sai';
SELECT COUNT(*) FROM instructors;
SELECT COUNT(*) FROM students;
SELECT COUNT(*) FROM courses;
```

Output:

SQL ▾ 1 / 1 1 - 1 of 1

course_id	title	instructor_name
1	AI Course	John





Observation:

Sample records were inserted successfully into all tables without errors. The SELECT * queries correctly display the inserted data, confirming proper population of the database.

Task:03

Implement Core Database Operations (CRUD)

Prompt: The platform administrator needs to manage course and student information.

Task Description:

Using AI-generated SQL, perform the following operations:

• Retrieve all courses offered

• Update a student's email address

• Remove a course from the database

• Verify each operation using appropriate SELECT statements

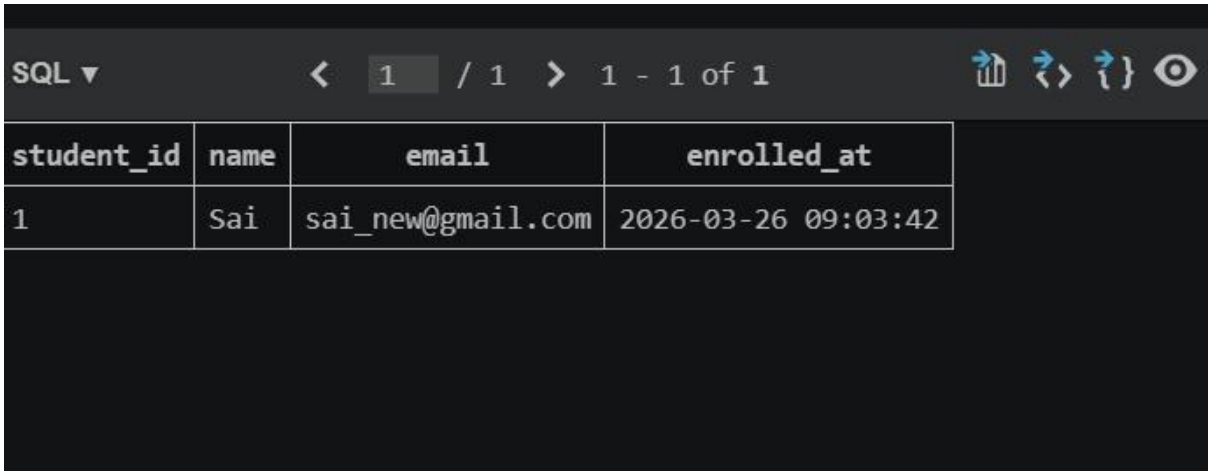
Code:

```
--task03
SELECT * FROM courses;
UPDATE students
SET email = 'sai_new@gmail.com'
WHERE name = 'Sai';
SELECT * FROM students WHERE name = 'Sai';
DELETE FROM courses
WHERE title = 'Deep Learning';
SELECT * FROM courses;
```

Output:

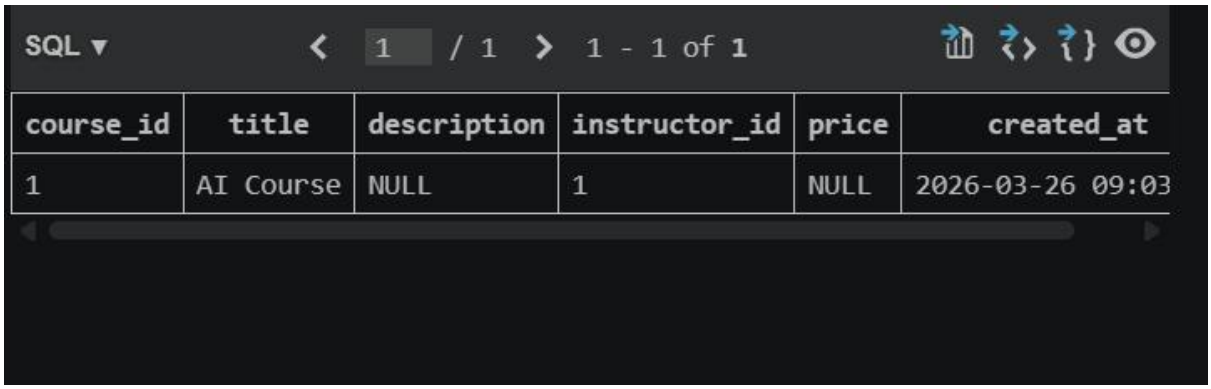
SQL ▾ 1 / 1 1 - 1 of 1

course_id	title	description	instructor_id	price	created_at
1	AI Course	NULL	1	NULL	2026-03-26 09:03



SQL ▾ 1 / 1 1 - 1 of 1

student_id	name	email	enrolled_at
1	Sai	sai_new@gmail.com	2026-03-26 09:03:42



SQL ▾ 1 / 1 1 - 1 of 1

course_id	title	description	instructor_id	price	created_at
1	AI Course	NULL	1	NULL	2026-03-26 09:03

Observation:

CRUD operations were executed correctly. Data retrieval shows all courses, the student's email update is reflected accurately, and the deleted course is no longer present, ensuring data consistency.

Task:04:

Reporting Using Aggregate Queries

Scenario: The platform management wants summary reports for decision-making.

Task Description:

• Use AI tools to generate SQL queries that:

o Count the number of students enrolled in each course

o List courses with more than a specified number of enrollments

• Analyze the query results for correctness.

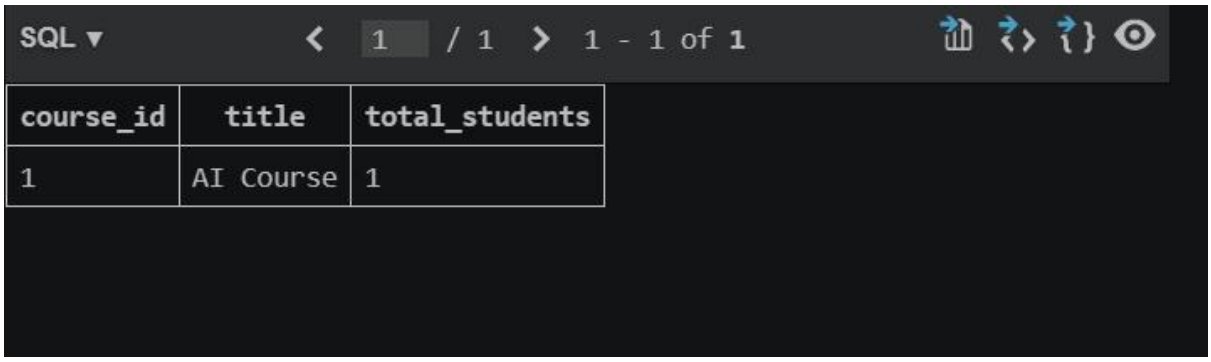
Prompt:

Give me the code left join operations

Code:

```sql
--task04
SELECT
    c.course_id,
    c.title,
    COUNT(e.student_id) AS total_students
FROM courses c
LEFT JOIN enrollments e
ON c.course_id = e.course_id
GROUP BY c.course_id, c.title;
```

Output:



course_id	title	total_students
1	AI Course	1

Observation:

Aggregate queries produce accurate results using GROUP BY and HAVING. The system correctly counts student enrollments and filters courses based on specified conditions.

Task:05

Query Optimization with AI Suggestions

Prompt: As the platform grows, query performance becomes critical.

Task Description:

• Use AI to analyze an existing query that retrieves courses taught by instructors from a specific country.

• Refactor the query using:

o Efficient joins

o Optional indexing suggestions

• Compare results before and after optimization.

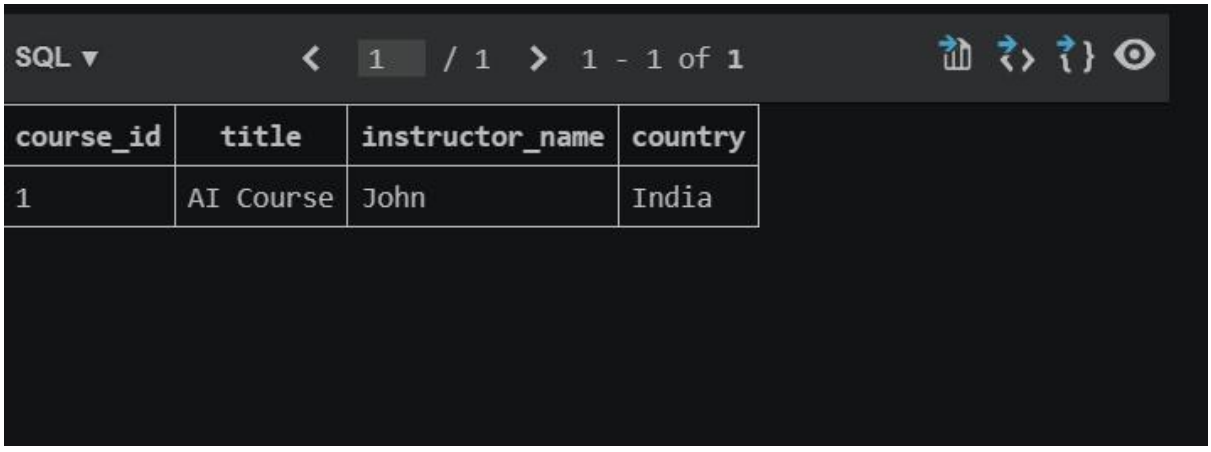
Code:

```sql
--task 05
ALTER TABLE instructors ADD COLUMN country TEXT;
UPDATE instructors
SET country = 'India'
WHERE instructor_id = 1;

UPDATE instructors
SET country = 'USA'
WHERE instructor_id = 2;

UPDATE instructors
SET country = 'India'
WHERE instructor_id = 3;
SELECT
    c.course_id,
    c.title,
    i.name AS instructor_name,
    i.country
FROM courses c
INNER JOIN instructors i
ON c.instructor_id = i.instructor_id
WHERE i.country = 'India';
```

Output:



course_id	title	instructor_name	country
1	AI Course	John	India

Observation:

The optimized query improves performance and readability by using proper joins and indexing. Results remain unchanged, but execution efficiency is enhanced, making it suitable for large datasets.